

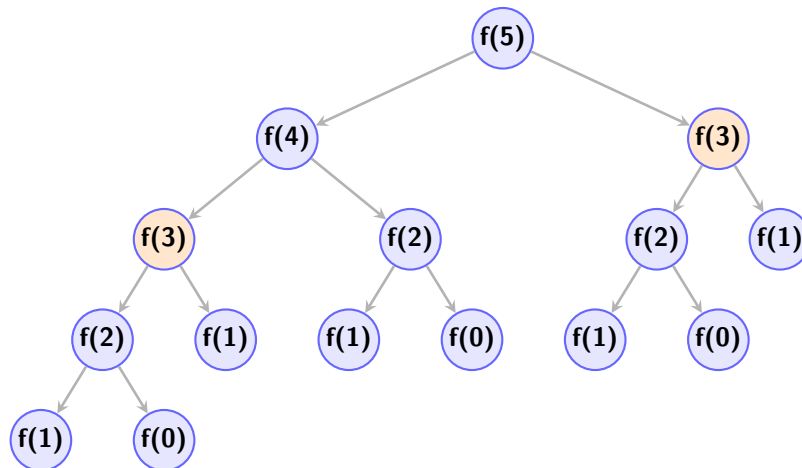
Dynamic Programming

Introduccion

Para introducirnos en programacion dinamica podemos tomar en cuenta el ejemplo mas conocido, el cual es la sucesion de fibonacci, para ello tenemos la solucion de forma recursiva:

$$f(n) = \begin{cases} 0, & \text{si } n = 0 \\ 1, & \text{si } n = 1 \\ f(n-1) + f(n-2), & \text{si } n > 1 \end{cases} \quad (1)$$

Pero esta solucion es muy poco optima para casos grandes ya que su complejidad es exponencial, es decir $O(2^n)$, por lo cual revisaremos cual es su mayor problema:



Como se puede ver el mayor problema de la recursion es que los calculos se repiten, podemos ver que $f(3)$ se calcula dos veces, para numeros mas grandes estos calculos repetidos se multiplican, para ello podemos utilizar un vector el cual guarde los datos ya calculados:

```
1 #define ll long long
2 vector<ll>memo(N+1,-1);
3 ll fib(ll n){
4     if(memo[n]!=-1) return memo[n];
5     if(n==0||n==1) return n;
6     return memo[n]=fib(n-1)+fib(n-2);
7 }
```

Utilizando la memorizacion podemos reducir la complejidad exponencial en lineal, es decir $O(n)$. Al igual que la memorizacion podemos usar otros dos conceptos de la programacion dinamica:

Push

Para entender esta parte de programación dinámica podemos tomar en cuenta el ejemplo de fibonacci de nuevo, tomemos en cuenta el elemento $fib(i)$, si nos damos cuenta este número contribuye a la suma de los dos siguientes elementos es decir $fib(i+1)$ y $fib(i+2)$.

```
1 void push_fib(){
2     vector<ll>dp(N+2,0);
3     dp[0]=0;
4     dp[1]=1;
5     for(int i=0;i<N;i++){
6         if(i+1<=N) dp[i+1]+=dp[i];
7         if(i+2<=N) dp[i+2]+=dp[i];
8     }
9 }
```

Un ejemplo donde el enfoque *Push* brilla es el problema de los saltos: Si una rana está en la posición i y puede saltar a $i+1$, $i+2$ o $i+3$, es mucho más natural .empujar. el valor actual a los tres destinos posibles que intentar calcular quiénes llegan al destino actual.

```
1 vector<ll>dp(N+4,0);
2 dp[0]=1;
3 for(int i=0;i<N;i++){
4     if(dp[i]==0) continue;
5     dp[i+1]+=dp[i];
6     dp[i+2]+=dp[i];
7     dp[i+3]+=dp[i];
8 }
```

Pull

El enfoque *Pull* es el más intuitivo y común en programación dinámica. En este caso, nos paramos en el estado actual i y "traemos"(pull) los resultados de subproblemas que ya han sido resueltos previamente para calcular el valor presente.

```
1 void pull_fib() {
2     vector<ll>dp(N+1,0);
3     dp[0]=0;
4     dp[1]=1;
5     for(int i=2;i<=N;i++) {
6         dp[i]=dp[i-1]+dp[i-2];
7     }
8 }
```